

Projet d'Introduction aux Sciences des Données

Classification d'Éoliennes sur Images Satellites

Étudiants : Amélie de Maillard et Mathis Rita

1 Introduction

Ce projet a pour objectif de développer un modèle de classification capable de détecter automatiquement la présence d'une éolienne dans des images satellites couleur de taille $3 \times 128 \times 128$. Pour cela, plusieurs modèles de réseaux de neurones ont été implémentés en PyTorch. Le jeu de données mis à disposition comprend 32 000 images d'entraînement ainsi qu'un jeu de validation séparé permettant d'évaluer la capacité de généralisation du modèle. L'évaluation finale sera réalisée sur un ensemble de 5 000 images inédites. Le travail demandé consiste à concevoir, entraîner et sélectionner un modèle performant pour détecter la présence (target) ou absence (other) d'une éolienne sur une image.

2 Méthodologie et modèles explorés

Afin de comprendre progressivement les mécanismes d'apprentissage et d'évaluer différentes approches, plusieurs modèles ont été testés au cours du projet.

Le premier modèle étudié fut une régression logistique. Cette approche simple permettait de mettre en place la chaîne de traitement complète, depuis la lecture des données jusqu'au calcul de l'accuracy, tout en fournissant une première base de comparaison. Cependant, ce modèle reste linéaire et ne tient pas compte de la structure spatiale des images. Les performances observées étaient limitées et ont rapidement montré la nécessité de passer à des architectures capables d'extraire automatiquement des motifs visuels pertinents.

La seconde étape a consisté à implémenter un réseau de neurones convolutionnel conçu manuellement. Ce modèle incluait plusieurs couches de convolution suivies de max-pooling et se terminait par une partie fully-connected. Par rapport à la régression logistique, les performances se sont nettement améliorées, ce qui confirme l'efficacité des convolutions pour exploiter la nature spatiale de l'image. Toutefois, au fil de l'entraînement, le modèle a montré des signes évidents de surapprentissage : la perte diminuait continuellement sur l'ensemble d'entraînement, alors qu'elle stagnait ou augmentait sur la validation. Il a donc été nécessaire d'augmenter la capacité de généralisation du réseau.

Les images étant organisées en deux dossiers (train, validation), des techniques de régularisation ont été introduites au cours des modèles que nous avons essayé :

- La première fut la data augmentation, incluant des retournements horizontaux aléatoires (RandomHorizontalFlip), des rotations (RandomRotation) et des variations de luminosité (ColorJitter). Ces transformations permettent au modèle de voir des versions plus variées des images d'entraînement et limitent ainsi la mémorisation excessive des données.
- La deuxième est la normalisation suivant les statistiques (ImageNet), ce qui facilite l'apprentissage dans le cadre de modèles pré-entraînés.

Ces différentes améliorations ont permis de stabiliser le comportement du modèle, de réduire l'écart entre l'entraînement et la validation, et d'obtenir des performances nettement plus robustes.

De plus, l'ensemble des données a été chargé via des DataLoader utilisant un batch de taille 32, ce qui permet un entraînement efficace sur GPU tout en maîtrisant l'occupation mémoire, y compris sur le grand jeu de données.

3 Modèle final : ResNet18 pré-entraîné

Bien que le CNN développé manuellement offrait des performances convenables, nous avons souhaité exploiter un modèle plus profond et bénéficiant d'une pré-formation sur une base de données massive. C'est dans ce cadre que nous avons adopté un ResNet18 pré-entraîné sur ImageNet. Cette technique, appelée transfert learning, permet de réutiliser des caractéristiques générales déjà apprises (bords, textures, structures) et d'adapter ensuite le modèle à notre tâche spécifique en ne réentraînant qu'une partie du réseau.

Dans notre cas, nous avons conservé l'ensemble des poids pré-entraînés, puis remplacé uniquement la couche fully-connected finale par une couche linéaire de sortie à deux neurones correspondant aux deux classes. Le reste du réseau est initialisé avec les poids pré-entraînés puis entièrement affiné (fine-tuning) sur notre jeu de données, avec un taux d'apprentissage faible afin de ne pas dégrader les poids préalablement acquis. L'entraînement est réalisé avec un batch de taille 32 sur des images en 128×128 , en utilisant l'optimiseur AdamW. Cet optimiseur a été choisi pour sa stabilité, et un scheduler a permis de réduire progressivement le taux d'apprentissage pour affiner les dernières étapes d'optimisation. Afin de sélectionner le meilleur modèle, un mécanisme de sauvegarde automatique (checkpoint) a été ajouté : à chaque epoch, on enregistrerait les nouveaux poids si l'accuracy sur la validation s'améliorait ou si à accuracy égale, la perte diminuait.

Cette approche s'est révélée particulièrement efficace. Les performances observées sont nettement supérieures à celles obtenues avec les modèles précédents, et la courbe de validation montre une bien meilleure généralisation. Le grand jeu de données prenant énormément de temps à charger, il était plus efficace de trouver comment améliorer le modèle avec le petit jeu de données (Data_Small) pour commencer. Le modèle final atteint une accuracy de validation de l'ordre de 98% pour le petit jeu de données. Afin de nous assurer que le modèle n'était pas trop biaisé par la taille, nous avons également vérifié l'accuracy sur la partie validation du grand jeu de données avec ces poids et les performances étaient légèrement moins bonnes (autour de 93%) mais toujours satisfaisantes. Les performances étant vraiment bonnes, nous avons commencé à entraîner le modèle sur le grand jeu de données. Nous souhaitions faire 5 epochs mais le programme cessait de fonctionner avant la fin. Pour régler ce problème tout en conservant une bonne performance, nous avons enregistré les poids calculés à chaque epoch dans le drive. De cette manière, nous pouvions savoir l'accuracy des poids enregistrés tout en cherchant à les améliorer, sans tout perdre en cas de déconnexion. Au final, l'accuracy de validation atteint environ 97.5% après 3 epochs d'entraînement. Cette performance, associée à un comportement stable entre l'entraînement et la validation, fait de ResNet18 le meilleur modèle testé dans ce projet.

4 Conclusion

Ce projet nous a permis d'explorer de manière progressive différentes approches de classification d'images, depuis les modèles linéaires jusqu'aux architectures profondes pré-entraînées. La construction d'un CNN a permis de comprendre les mécanismes fondamentaux de l'apprentissage profond, tandis que l'ajout de régularisation et de data augmentation a mis en évidence les enjeux liés à la généralisation. L'utilisation finale de ResNet18 en transfert learning s'est avérée la solution la plus performante et la plus stable, confirmant l'importance des modèles pré-entraînés pour les tâches de vision complexes.